NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
Department of Computer Science & Information Technology
CT-260 | Object Oriented Programming | Spring 2026

COMPLEX COMPUTING PROBLEM (CCP) REPORT

Password Security Suite

A Console-Based C++ Application
Demonstrating Core OOP Principles Applied to Real-World Cybersecurity

Course	CT-260 — Object Oriented Programming
Semester	2nd Semester	Spring 2026
Section	B
Submitted To	Dr. Murk Marvi
Group Members	Areefa Samar	Arbish
Roll Numbers	CT-25062	CT-25053
Submission Date	11-May-2026

TABLE OF CONTENTS

1. Problem Statement

1.1 Background

Most users still rely on weak, reused, or leaked passwords directly enabling identity theft and data breaches. Commercial password managers are effective but often require subscriptions or cloud access. A local, console-based tool that handles password generation, evaluation, breach detection, and secure storage fills this gap without requiring any internet connectivity or paid services.

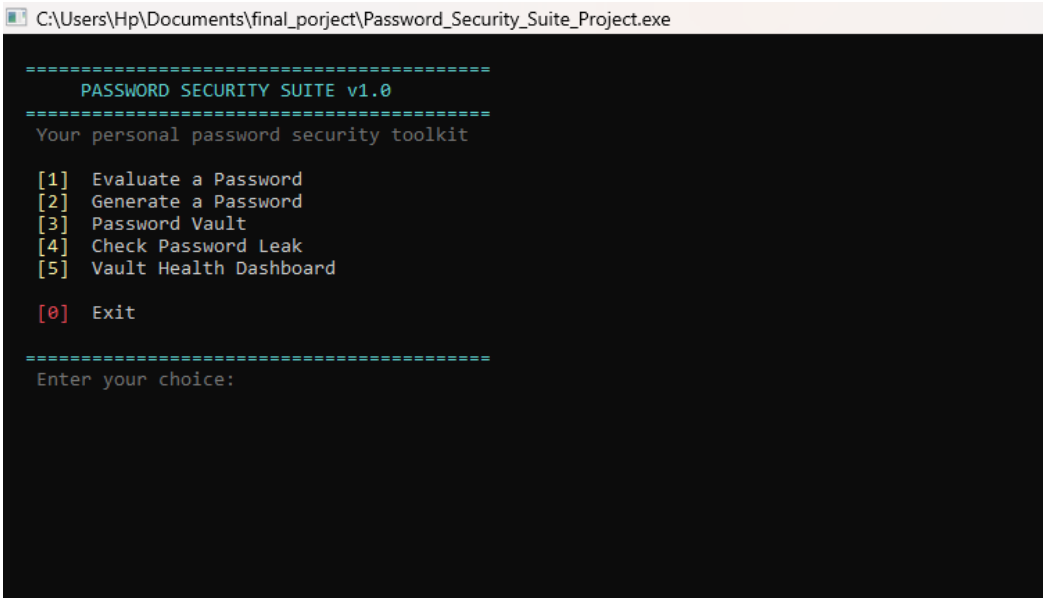

Figure 1 — Password Security Suite main menu

1.2 Computing Challenge

The system must perform several non-trivial computing tasks simultaneously:

- Heuristic strength analysis covering length, character variety, common patterns, and dictionary matching.
- Password generation via two algorithms random character selection and word-phrase composition.
- Encrypted local vault with master-password authentication and brute-force lockout after three failures.
- Offline breach detection against a local dataset of known compromised passwords.
- Health reporting by combining all subsystems into one consolidated analytical dashboard.

Why It Matters

Password-related breaches account for over 80% of hacking incidents globally. Password Security Suite is a fully offline, free alternative to commercial password managers built entirely from first principles using C++ and OOP.

1.3 OOP Relevance

Every feature maps naturally to a class with clear, single responsibilities. Inheritance, composition, and aggregation arise from the problem itself not forced in to satisfy a checklist. This makes the project a genuine demonstration of OOP applied to a real engineering problem rather than an academic exercise.

2. OOP Principles Identification

All four pillars of OOP appear naturally in this project. Each is justified below with the specific classes and design decisions that demonstrate it.

2.1 Encapsulation

Encapsulation means bundling data and the methods that operate on it inside a class, and restricting external access to internal state.

In PasswordEvaluator, the private fields (password, score, level) and six private checking methods (checkLength(), checkUppercase(), checkDigits(), checkSymbols(), checkCommonPatterns(), checkLeakedList()) are completely hidden. The public interface only exposes setPassword(), evaluate(), displayReport(), and read-only getters preventing external code from bypassing the scoring logic or reading a stale score.

In AuthGuard, all security-critical fields (failCount, isLocked, masterHash, lockTime) are private. External code can only call verifymasterPassword() and getLockStatus() it cannot reset the fail counter or clear the lock directly, which would defeat the brute-force protection.

2.2 Inheritance

Inheritance allows a derived class to acquire the properties and behaviours of its base class, promoting code reuse and a natural IS-A hierarchy.

PasswordGenerator is the abstract base class, declaring the pure virtual generate() and holding shared configuration (length, useUppercase, useDigits, useSymbols, avoidLookAlikes) with validated setters. Both RandomGenerator and PhraseGenerator inherit from it reusing all the setter/getter infrastructure without reimplementing it. Each subclass IS-A PasswordGenerator and must provide its own generate() implementation.

2.3 Polymorphism

Polymorphism allows one interface to have multiple implementations, resolved at runtime based on the actual type of the object.

Both RandomGenerator and PhraseGenerator override generate(). When invoked through a PasswordGenerator pointer or reference, C++ dispatches to the correct subclass at runtime. RandomGenerator produces a character-pool password; PhraseGenerator produces a word-based passphrase. Same call different output depending entirely on the object's runtime type.

2.4 Abstraction

Abstraction means hiding complex implementation details behind a clean, simple interface.

PasswordGenerator cannot be instantiated directly because generate() is pure virtual. Users must work through a concrete subclass, but always via the common PasswordGenerator interface they never need

to know how the character pool is built or how words are selected. Similarly, PasswordEvaluator callers simply call evaluate() without knowing how the score is computed or how crack time is estimated.

OOP Principle	Where It Appears	Specific Classes
Encapsulation	Private data + restricted public interface	PasswordEvaluator, AuthGuard, PasswordVault
Inheritance	Base class + concrete subclasses sharing state	PasswordGenerator → RandomGenerator, PhraseGenerator
Polymorphism	Virtual generate() dispatched at runtime	RandomGenerator, PhraseGenerator override generate()
Abstraction	Pure virtual base; implementation hidden	PasswordGenerator (abstract), PasswordEvaluator

Table 1 — OOP principles mapping — where each pillar appears in the system

3. System Design & UML Class Diagram

3.1 Architecture Overview

The system splits into two independent subsystems the Input & Analysis Engine (Partner A) and the Storage & Reporting Engine (Partner B) that communicate through pre-agreed public interfaces. The overall design is layered: user interaction on top, feature-level classes in the middle, and file I/O at the base.

3.2 Class Responsibilities

Class	Role	Responsibilities
PasswordGenerator	Abstract Base	Declares pure virtual generate(). Stores shared config: length, character-type flags, look-alike avoidance. Cannot be instantiated.
RandomGenerator	Concrete Subclass	Inherits from PasswordGenerator. Overrides generate() to build a random character-pool password. Private buildCharPool() assembles the allowed set.
PhraseGenerator	Concrete Subclass	Inherits from PasswordGenerator. Overrides generate() to produce a word-based passphrase (e.g. Blue$Tiger$Runs49). Configurable word count and separator.
PasswordEvaluator	Standalone Class	Evaluates strength across six criteria. Computes 0-100 score and maps to StrengthLevel enum. Provides displayReport() for formatted output.
PasswordEntry	Value Object	Stores one vault record: label, encrypted password, date added. Validates date format on construction.
AuthGuard	Security Controller	Verifies master password. Tracks failed attempts; enforces 30-second lockout after three failures. All state is private.
MasterPasswordFile	Utility (Static)	Manages the master_passwords.txt file. Static methods to check existence, save credentials, and verify a supplied password.
PasswordVault	Aggregate / Manager	Owns vector<PasswordEntry> and composes AuthGuard. Handles create, open, save, delete, and search. XOR-encrypts passwords before file write.
LeakChecker	Standalone Checker	Loads a local leaked-password file into a vector<string> at construction. Exposes isLeaked() for case-insensitive lookup.
HealthDashboard	Report Aggregator	Combines PasswordVault, LeakChecker, and PasswordEvaluator to scan all vault entries, tally weak/compromised passwords, and display a health report.

Table 2 — Class responsibilities — all ten classes with roles and descriptions

3.3 Class Relationships

From	Relationship	To	Why
RandomGenerator	Inheritance	PasswordGenerator	IS-A PasswordGenerator
PhraseGenerator	Inheritance	PasswordGenerator	IS-A PasswordGenerator
PasswordVault	Composition (owns)	AuthGuard	AuthGuard lifecycle tied to Vault
PasswordVault	Composition (owns)	PasswordEntry[ ]	Entries created & destroyed within Vault
HealthDashboard	Aggregation (uses)	PasswordVault	Dashboard uses Vault but does not own it
HealthDashboard	Aggregation (uses)	LeakChecker	Dashboard uses LeakChecker without owning it
HealthDashboard	Aggregation (uses)	PasswordEvaluator	Dashboard uses Evaluator without owning it
PasswordEvaluator	Association (reads)	leaked_passwords.txt	Evaluator reads file via checkLeakedList()
LeakChecker	Association (reads)	leaked_passwords.txt	LeakChecker loads the file at construction
MasterPasswordFile	Association (r/w)	master_passwords.txt	Static class manages vault credential persistence

Table 3 — Class relationships — inheritance, composition, aggregation, and association

3.4 UML Notation Key

Symbol	Meaning	Example in this Project
▲ (hollow arrow up)	Inheritance / Generalisation	RandomGenerator → PasswordGenerator
♦ (filled diamond)	Composition (strong ownership)	PasswordVault ♦→ AuthGuard
◇ (hollow diamond)	Aggregation (weak ownership)	HealthDashboard ◇→ PasswordVault
——► (solid arrow)	Association (uses / reads)	PasswordEvaluator →→ leaked_passwords.txt

«abstract»	Abstract class stereotype	PasswordGenerator
«static»	Static method stereotype	MasterPasswordFile methods
# prefix	Protected access modifier	PasswordGenerator attributes
- prefix	Private access modifier	AuthGuard::failCount
+ prefix	Public access modifier	All public methods

Table 4 — UML notation key — symbols used in the class diagram

3.5 UML Class Diagram

The diagram below illustrates the full class hierarchy, relationships, and key attributes/methods for each class in the system. Standard UML notation is used throughout.

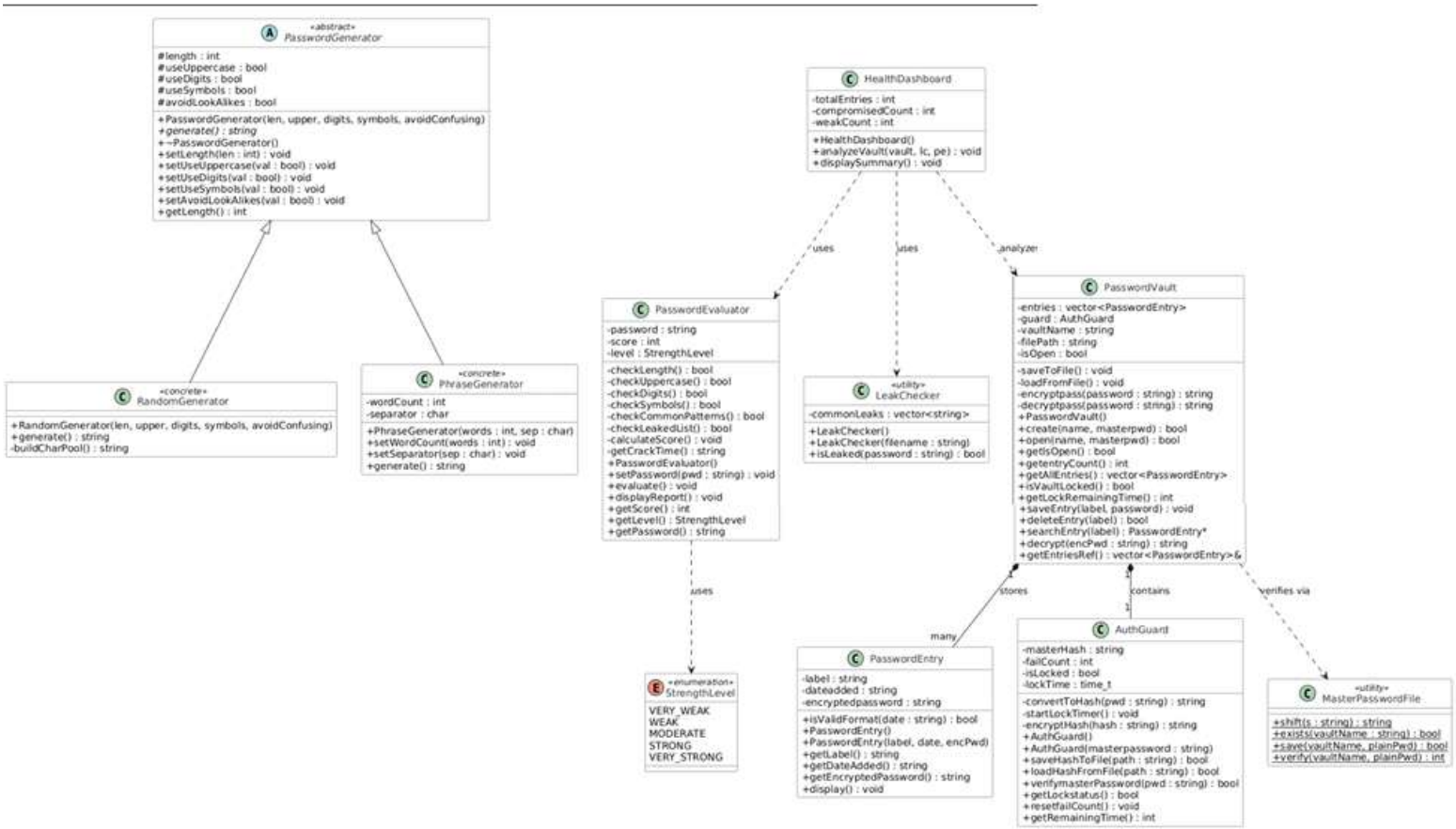

Figure 2 — UML Class Diagram — full system hierarchy

4. Solution Definition

4.1 Feature 1 — Password Evaluator

PasswordEvaluator receives a plaintext password via setPassword(). Calling evaluate() computes a 0-100 score using six private checks, each encapsulated inside the class and invisible to external code.

Criterion	Points	Logic
Length	+5 to +30	>=8: +12 | >=12: +20 | >=16: +25 | >=20: +30
Lowercase present	+10	At least one lowercase character
Uppercase present	+15	At least one uppercase character
Digits present	+15	At least one numeric digit
Symbols present	+20	At least one symbol from the defined set
Common pattern found	-20	Deducted if patterns like '123', 'abc', or 'qwerty' detected
Found in leaked list	Score -> 0	Score forced to 0; password is known compromised

Table 5 — Password Evaluator scoring criteria — points awarded per criterion

Score mapping to StrengthLevel: VERY_WEAK (0-19), WEAK (20-39), MODERATE (40-59), STRONG (60-79), VERY_STRONG (80-100). displayReport() shows the score, an ASCII progress bar, a crack-time estimate, per-criterion pass/fail marks, and targeted improvement suggestions.

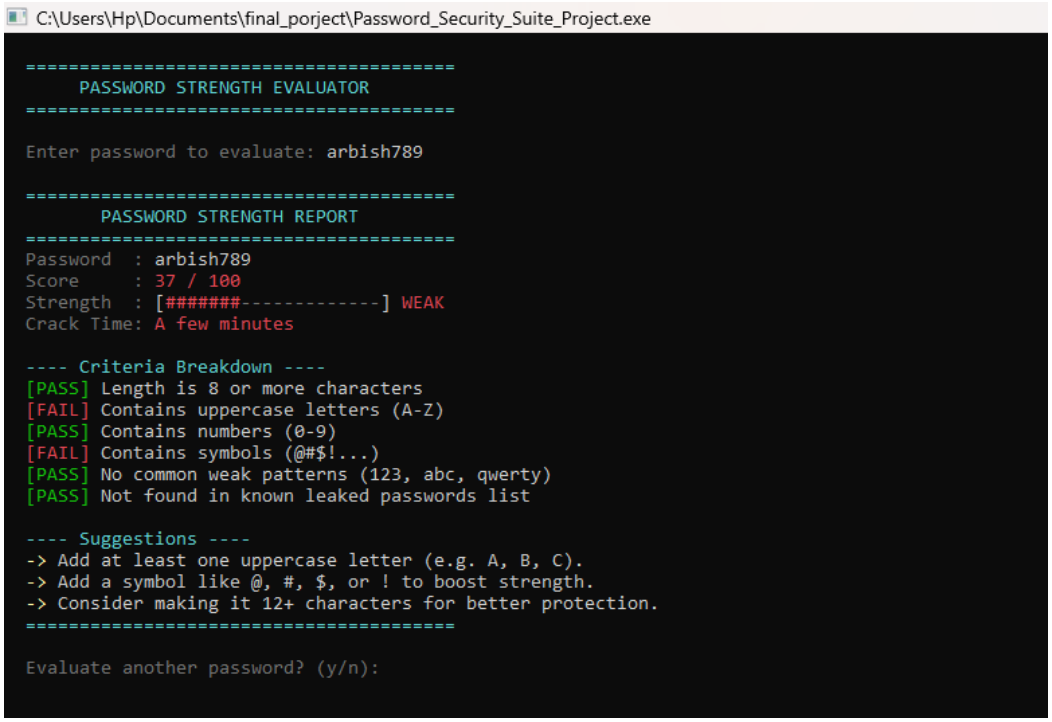

Figure 3 — Password Evaluator output for a weak password

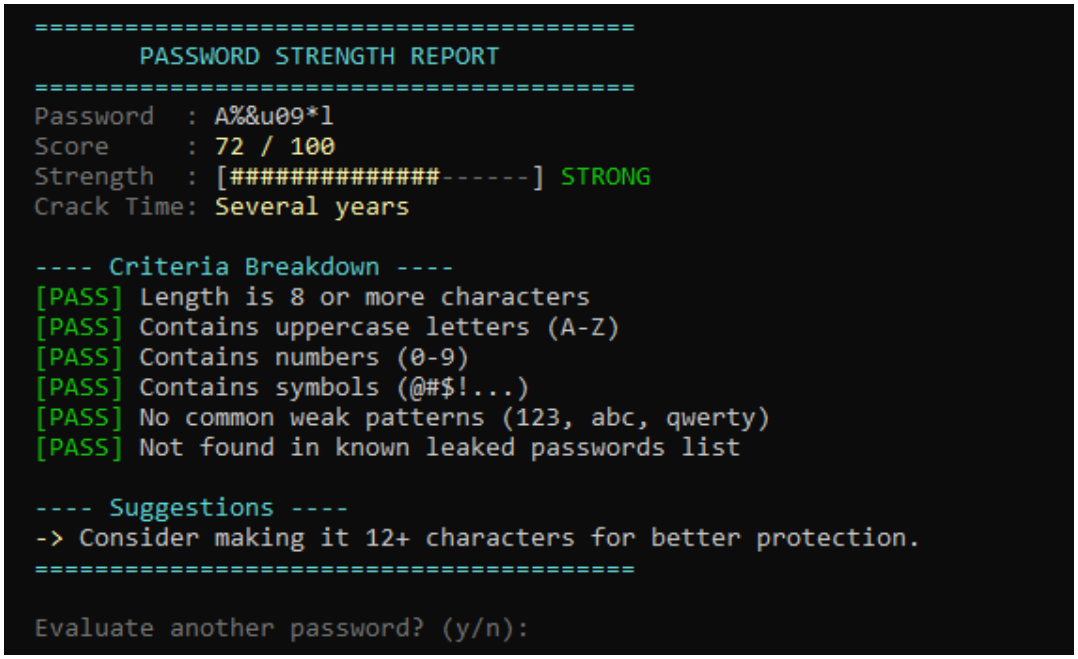

Figure 4 — Password Evaluator output for a strong password

4.2 Feature 2 — Password Generator

The user selects Phrase Mode (PhraseGenerator) or Random Mode (RandomGenerator). The program asks a sequence of questions tailored to the selected mode before generating.

Random Mode — buildCharPool()	pool <- lowercase alphabet if useUppercase -> add A-Z if useDigits -> add 0-9 if useSymbols -> add @#$!%&* for i in 0 to length: password += pool[rand() % pool.size()]
Phrase Mode — generate()	for i in 0 to wordCount: pick random word from 56-word list if i < wordCount-1: append separator append random 2-digit number (10-99) Example: Blue$Tiger$Runs49

Table 6 — Password Generator algorithm pseudocode — Random Mode and Phrase Mode

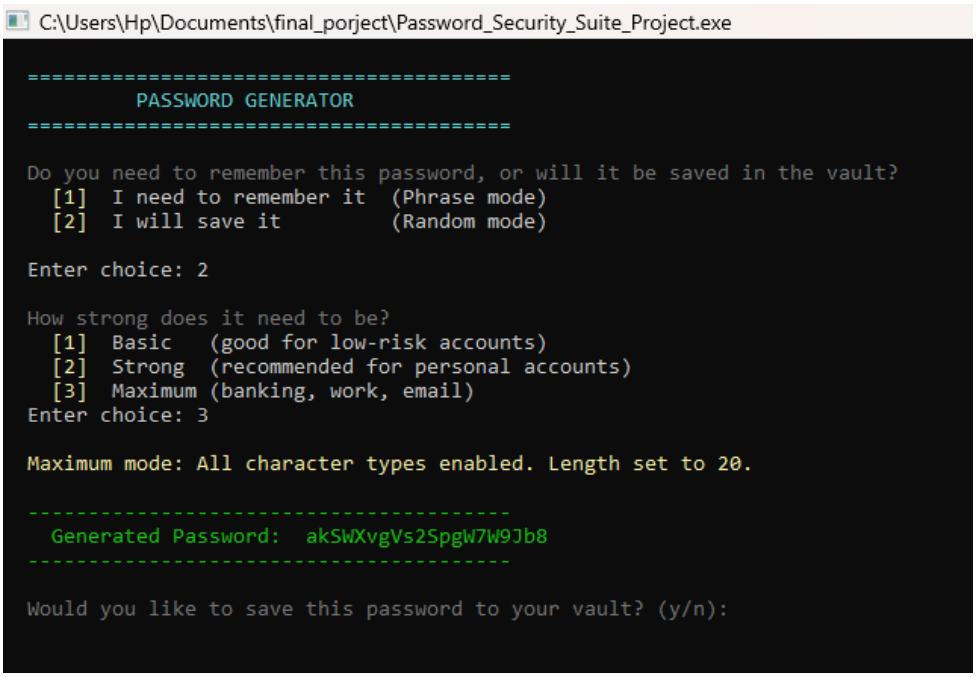

Figure 5 — Password Generator in Random Mode

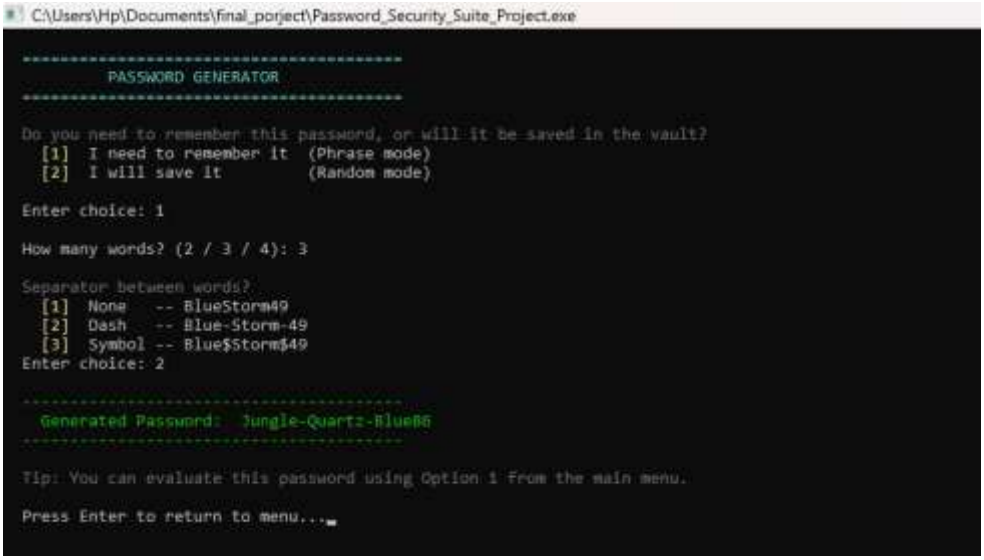

Figure 6 — Password Generator in Phrase Mode

4.3 Feature 3 — Password Vault

PasswordVault has two lifecycle phases: creation and access. On creation, the master password is hashed and saved to master_passwords.txt via MasterPasswordFile::save(). On access, MasterPasswordFile::verify() compares the stored entry. AuthGuard tracks failures and enforces a 30-second lockout after three wrong attempts. Each stored password is XOR-encrypted with key 9 before being written to a per-vault CSV file. The same XOR operation reverses the encryption on read.

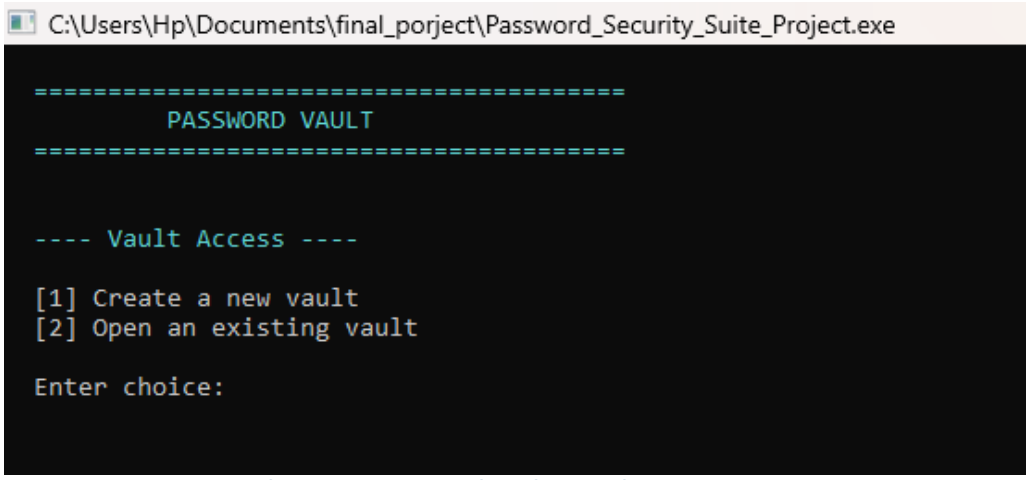

Figure 7 — Password Vault — vault access menu

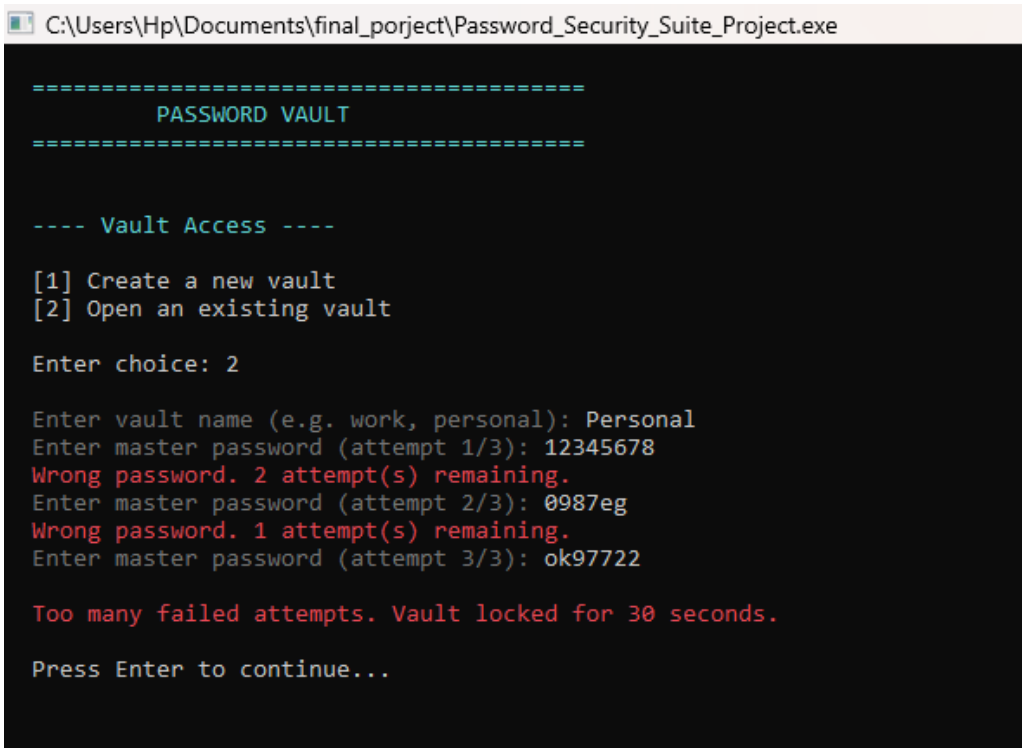

Figure 8 — Password Vault — brute-force lockout after three failed attempts

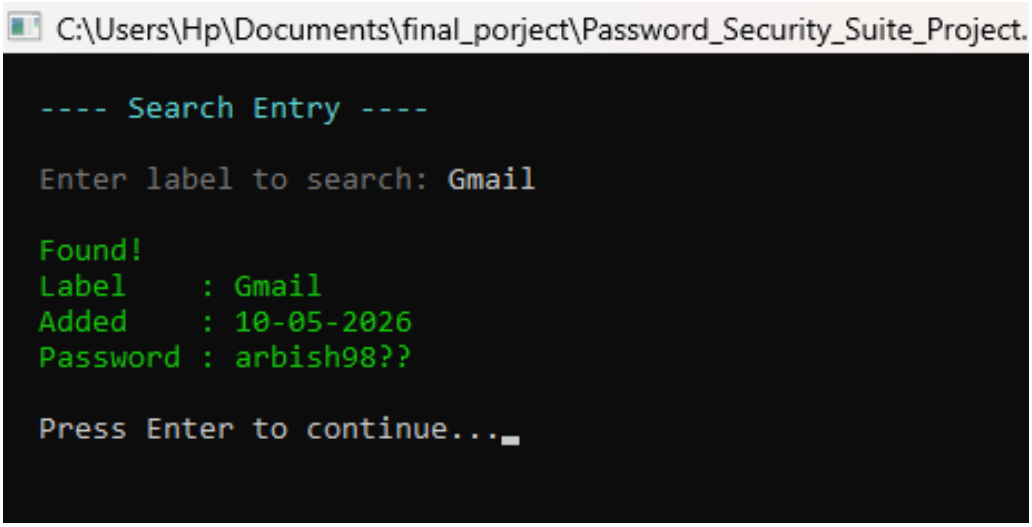

Figure 9 — Password Vault — search entry by label

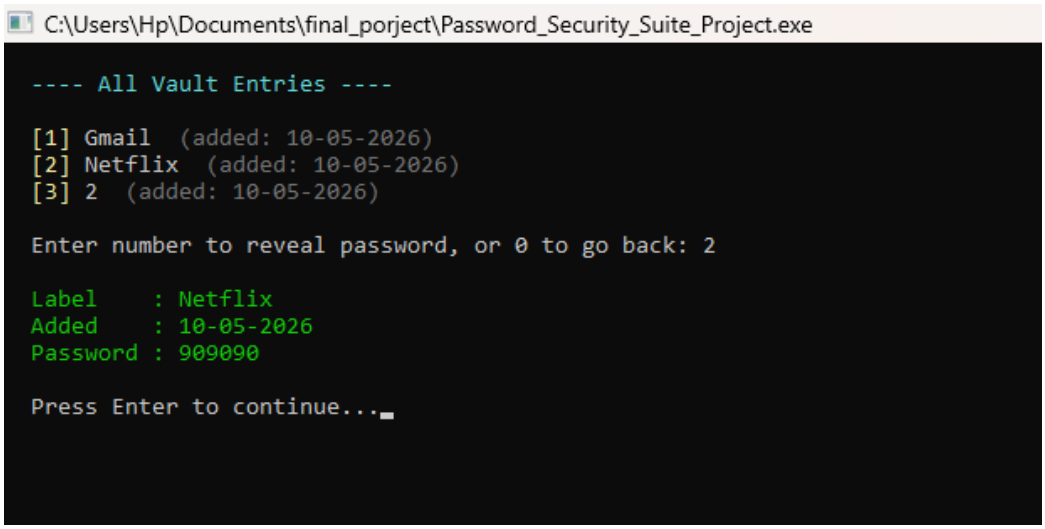

Figure 10 — Password Vault — view all entries and reveal password

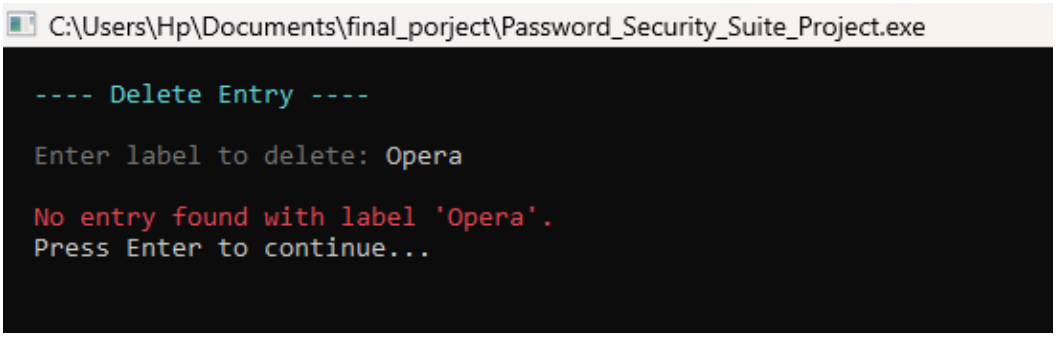

Figure 11 — Password Vault — delete entry

4.4 Feature 4 — Leaked Password Checker

LeakChecker is constructed with a filename. It reads the file line by line, normalises each entry to lowercase, and stores the list in a private vector<string>. isLeaked() converts the supplied password to lowercase and performs a linear search. The feature works entirely offline with a local breach dataset of 100+ known compromised passwords.

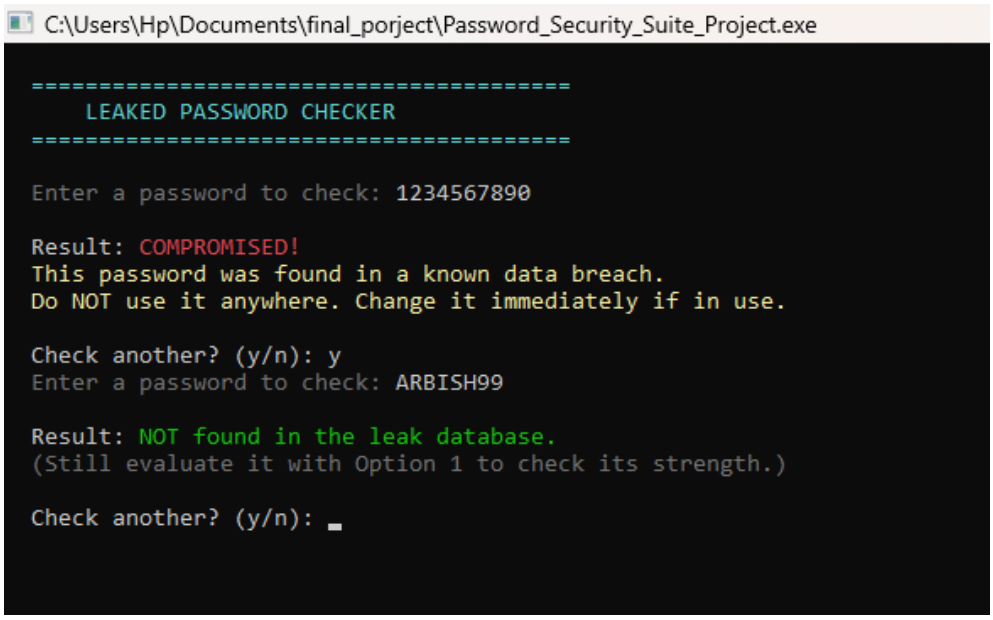

Figure 12 — Leaked Password Checker — compromised and safe results

4.5 Feature 5 — Health Dashboard

HealthDashboard::analyzeVault() takes references to a PasswordVault, LeakChecker, and PasswordEvaluator. It loops over every vault entry, decrypts the stored password, calls isLeaked() and evaluate(), and maintains two independent counters compromisedCount for confirmed leaked passwords and weakCount for low-scoring ones. The overall health percentage reported by displaySummary() is calculated from compromised entries only; weak passwords are surfaced as a separate warning, giving users a clear distinction between accounts requiring immediate action and those advisable to strengthen.

Integration Point

HealthDashboard demonstrates OOP composition in action it talks to PasswordVault, LeakChecker, and PasswordEvaluator purely through their public interfaces, with no knowledge of internal implementation details. This is the Facade pattern applied directly.

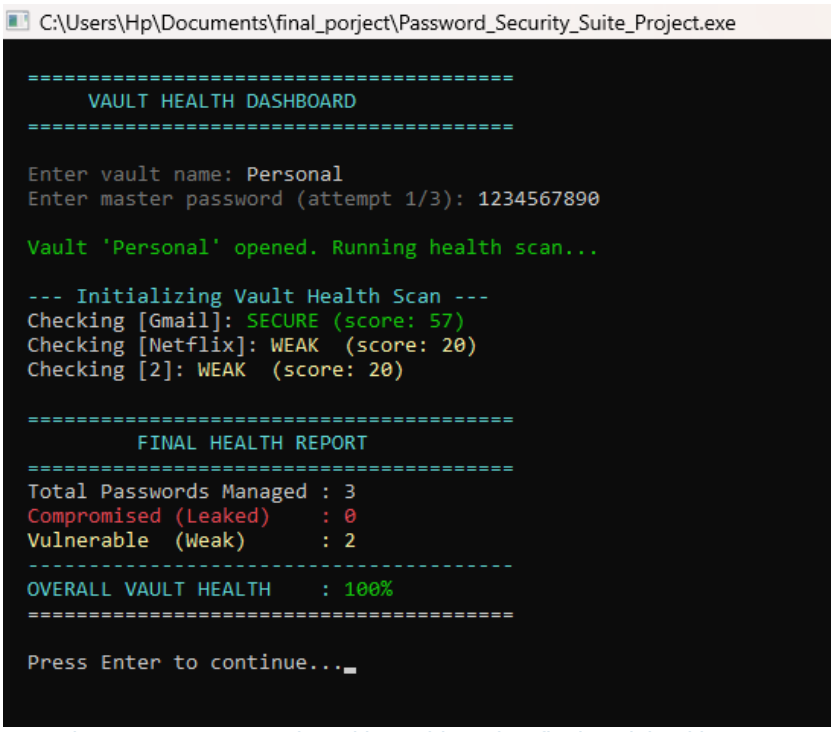

Figure 13 — Password Health Dashboard — final vault health report

5. Design Pattern Application

5.1 Template Method Pattern

Defines an algorithm's skeleton in a base class and defers specific steps to subclasses. PasswordGenerator declares generate() as pure virtual the skeleton. The concrete steps (how the character pool is built vs. how words are selected) are left entirely to RandomGenerator and PhraseGenerator. Callers invoke generate() on a PasswordGenerator reference without caring which subclass executes.

5.2 Strategy Pattern

Defines a family of interchangeable algorithms and selects one at runtime. RandomGenerator and PhraseGenerator are two interchangeable strategies. The user selects Phrase vs. Random at runtime; from that point, the rest of the program uses the common PasswordGenerator interface. Adding a new strategy (e.g. PINGenerator) requires zero changes to calling code.

5.3 Facade Pattern

Provides a simple interface over a complex subsystem. HealthDashboard hides the complexity of iterating entries, decrypting passwords, running evaluations, and checking leak databases behind just two calls analyzeVault() and displaySummary(). Users of HealthDashboard need no knowledge of the three subsystems it orchestrates.

5.4 Guard / Sentinel Pattern

All access to the vault is mediated through a single AuthGuard object that maintains fail count, lock status, and lock timer in one place. This prevents authentication logic from being scattered across the codebase and ensures the lockout cannot be bypassed by calling a different code path.

Pattern	Classes Involved	Benefit	Type
Template Method	PasswordGenerator, RandomGenerator, PhraseGenerator	Enforces consistent algorithm skeleton; defers implementation details to subclasses	Formal (GoF)
Strategy	RandomGenerator, PhraseGenerator via PasswordGenerator	Interchangeable algorithms selectable at runtime without changing calling code	Formal (GoF)
Facade	HealthDashboard over Vault, Evaluator, LeakChecker	Simplifies multi-subsystem orchestration behind a two-method interface	Formal (GoF)
Guard / Sentinel	AuthGuard inside PasswordVault	Centralises and enforces access-control policy; prevents bypass via alternative paths	Informal

Table 7 — Design pattern summary — pattern, classes involved, benefit, and type

6. Limitations & Future Work

6.1 Current Limitations

Limitation	Detail & Impact
XOR Encryption Weakness	Simple XOR with a fixed key is not cryptographically secure. A production system would use AES-256 or bcrypt.
Plain-text Leaked Database	The leaked-password file is plaintext easily inspected or modified. A production system would use a Bloom filter or hashed dataset.
No Salt on Master Password	convertToHash() produces a deterministic checksum. Without a random salt, identical passwords produce identical hashes, enabling precomputation attacks.
Linear Leak Search	isLeaked() performs a linear scan. For millions of entries this becomes too slow std::unordered_set would give O(1) average-case lookup.
Console-Only Interface	Non-technical users must navigate text menus. No GUI or web interface is provided.
Single-User, Single-Machine	All data is in local files. No sync, no multi-device access, and no backup mechanism.
No Password Age Tracking	The vault stores a date-added field but does not currently alert users when a password is stale (e.g. older than 90 days).
Static Word List	PhraseGenerator uses a hardcoded 56-word list. A larger external dictionary would produce significantly more entropy.

Table 8 — Current limitations — constraint, detail, and impact

6.2 Proposed Enhancements

- Replace XOR encryption with AES-256 via OpenSSL or a header-only C++ library.
- Replace the master password checksum with bcrypt or PBKDF2 with a random per-vault salt.
- Replace the linear leak search with std::unordered_set for O(1) average-case lookup.
- Add password expiry alerts flag vault entries in the Health Dashboard when date added exceeds a configurable threshold (e.g. 90 days).
- Implement a GUI using Qt, or a web-based interface, to improve usability for non-technical users.
- Add cloud-sync with end-to-end encryption so the vault is accessible across devices.
- Load the phrase word list from a configurable external file instead of a hardcoded array.
- Add categories and tags on PasswordEntry objects (Work, Banking, Social, etc.) with filter support in the vault menu.
- Add import/export for standard password-manager formats such as CSV and KeePass XML.

7. Team Contribution Table

Member Name	Roll Number	Section / Theme	Specific Contributions
Areefa Samar	CT-25062	Partner A — Input & Analysis Engine	**Feature 1: PasswordEvaluator** All six criteria, score calculation, crack time estimate, displayReport() **Feature 2: Password Generator** Abstract base class PasswordGenerator, RandomGenerator, PhraseGenerator, runPasswordGenerator()
Arbish	CT-25053	Partner B — Storage & Reporting Engine	**Feature 3: Password Vault** PasswordVault, AuthGuard, MasterPasswordFile, PasswordEntry — encryption, file I/O, lockout **Feature 4: LeakChecker** Offline breach detection against local dataset **Feature 5: HealthDashboard** Multi-class aggregation and full security report generation

Table 9 — Team contribution table — member names, roll numbers, and contributions

7.1 Individual Class Assignment

Class	Partner	OOP Concept	Files
PasswordGenerator	Partner A	Abstraction, Inheritance	PasswordGenerator.h / .cpp
RandomGenerator	Partner A	Inheritance, Polymorphism	PasswordGenerator.h / .cpp
PhraseGenerator	Partner A	Inheritance, Polymorphism	PasswordGenerator.h / .cpp
PasswordEvaluator	Partner A	Encapsulation, Abstraction	PasswordEvaluator.h / .cpp
PasswordEntry	Partner B	Encapsulation, Validation	PasswordEntry.h / .cpp
AuthGuard	Partner B	Encapsulation, Security Logic	AuthGuard.h / .cpp
MasterPasswordFile	Partner B	File I/O, Static Methods	PasswordVault.h / .cpp

PasswordVault	Partner B	Composition, File I/O, Arrays	PasswordVault.h / .cpp
LeakChecker	Partner B	Encapsulation, File I/O	LeakChecker.h / .cpp
HealthDashboard	Partner B	Aggregation, Facade Pattern	HealthDashboard.h / .cpp

Table 10 — Individual class assignment — class, partner, OOP concept, and files

8. Conclusion

Password Security Suite demonstrates that Object Oriented Programming is not simply a theoretical framework it is a practical engineering discipline that produces cleaner, safer, and more maintainable software when applied thoughtfully to real problems.

All four OOP pillars appear in this project not because the rubric requires them, but because the problem demands them. Encapsulation protects security-critical data from external tampering. Inheritance eliminates code duplication across two generator strategies. Polymorphism allows the main menu to call generate() without knowing or caring which mode the user selected. Abstraction prevents callers from depending on internal scoring or encryption logic that may change in future iterations.

Beyond the OOP principles, the project demonstrates three formal design patterns Template Method, Strategy, and Façade each justified by the structure of the problem rather than chosen arbitrarily. The AuthGuard class implements an informal Guard pattern that centralises all authentication logic and prevents it from being scattered across the codebase.

The five features password evaluation, generation, secure vault storage, breach detection, and health reporting form a coherent, end-to-end security tool. Each feature is independently testable, independently maintainable, and connected to the others only through clean public interfaces. This is precisely the kind of modular, extensible architecture that OOP is designed to produce.

The limitations acknowledged in Section 6 XOR encryption, linear search, no GUI are honest reflections of the project's scope as a second-semester console application. The proposed enhancements chart a clear path from this educational prototype toward a production-grade password manager, demonstrating awareness of real-world software engineering constraints beyond what a first implementation can address.

Summary

Password Security Suite is a fully offline, console-based C++ application that solves a genuine real-world problem using all four OOP pillars, three formal design patterns, robust input and exception handling, and a modular class architecture that could realistically be extended into a production tool.

9. References

[1] Eckel, B. (2000). *Thinking in C++, Volume 1: Introduction to standard C++* (2nd ed.). Prentice Hall.

[2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

[3] Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ primer* (5th ed.). Addison-Wesley.

[4] National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-63b

[5] Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

[6] Verizon. (2024). *2024 data breach investigations report*. Verizon Communications. https://www.verizon.com/business/resources/reports/dbir/

[7] OWASP Foundation. (2023). Password storage cheat sheet. Open Web Application Security Project. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[8] Hunt, T. (2019). Pwned passwords. HaveIBeenPwned. https://haveibeenpwned.com/Passwords

— End of Report —